机化 2201　黄钊伟　2022013106

实验六　单片机的扩展

（一）存储器的扩展

1.实验目的

（1）掌握并行存储器芯片引脚的功能及与单片机的连接方法；

（2）掌握对扩展存储器的编程方法。

2.实验设备

计算机、Keil μVision 软件、Proteus 软件。

3.实验内容

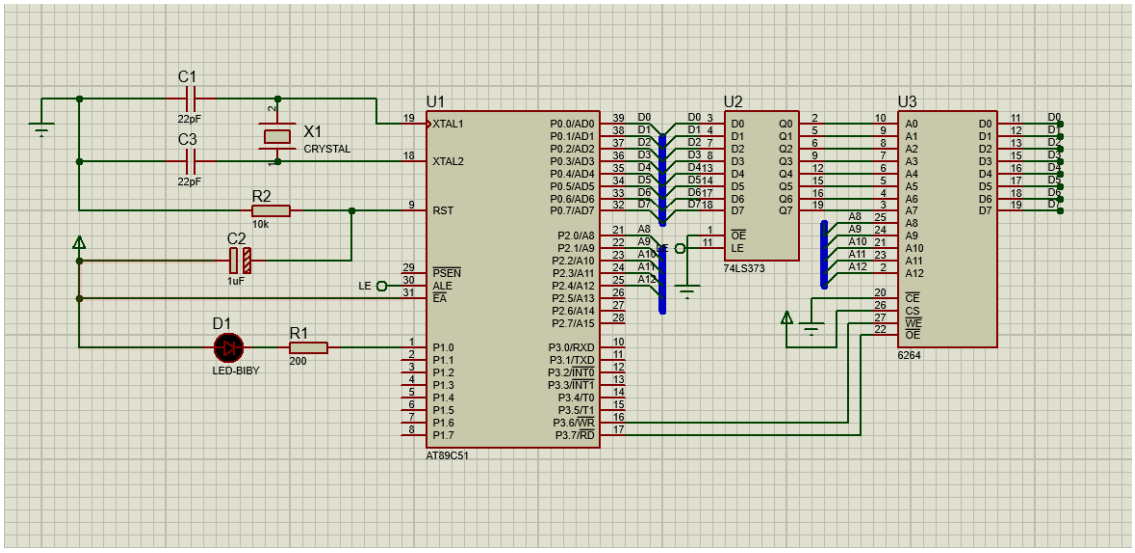
（1）数据存储器的扩展

给某 89C51 单片机扩展一片 6264，并实现将内部 RAM 中存放的字符串“Welcome to Northwest A&F University!”和“Welcome to College of Mechanical and Electronic Engineering!”，分别存放到外部 RAM 6264 中从 0 单元和 50H 单元开始的地方。当传送结束时点亮连接于 P1.0 口的 LED。

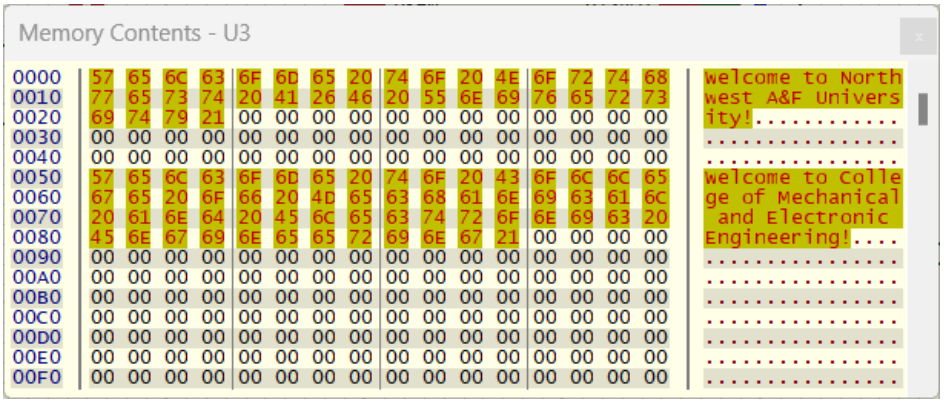
运行代码：

```c
#include <reg51.h>
#include <absacc.h>

#define uchar unsigned char
#define uint unsigned int

unsigned char idata table1[]="Welcome to Northwest A&F University!$";
unsigned char idata table2[]="Welcome to College of Mechanical and Electronic Engineering!$";
sbit led=P1^0;

void main()
{
  uint i=0;
  led=1;
  while(table1[i]!='$')
  {
    XBYTE[i]=table1[i];
    i++;
  }
  i=0;
  while(table2[i]!='$')
  {
    XBYTE[i+0x50]=table2[i];
    i++;
  }

  led=0;
  while(1);
}
```

电路图：



运行结果：



4.实验提示

（1）对于实验内容（2），在对 6264 扩展时一定保证扩展的 6264 从 0 单元开始。

（2）排阻在 Proteus 中的符号是 RESPACK，RESPACK 有 7 路（RESPACK-7）和 8 路（RESPACK-8）两种，对于任何一种，引脚 1 为公共端，接电源或地线。引脚 1 与其他引脚端有一个电阻。排阻的阻值根据需要选用。

5. 观察与分析

（1）分析实验内容（2）中所扩展的 6264 的地址范围，指出 6264 的容量是多少 kB？如果将 6264 换为 62128，则连线将如何修改？

6264 共有 13 根地址线，因此地址范围为 0x2000—0x3FFF，容量为 8kB，将 6264 换为 62128 后，共有 14 根地址线，因此需要额外接一根地址线。

（2）扩展程序存储器和数据存储器时所用的控制信号线有哪些？哪些是用于扩展程序存储器的，哪些又是用于扩展数据存储器的？

有 PSEN、EA、RD、WR 等，其中，PSEN 和 EA 用于扩展程序存储器，RD 与 WR 用于扩展数据存储器。

（3）执行汇编语言指令 MOVC A, @A+DPTR 时，哪个控制信号线有效（变为低电平）？执行 MOVX A，@DPTR 和 MOVX @DPTR，A 时又分别是哪些控制信号线有效？

执行汇编语言指令 MOVC A, @A+DPTR 时，PSEN 信号线有效，执行 MOVX A，@DPTR 和 MOVX @DPTR，A 时分别是 RD 与 WR 有效。

（二）I/O 口的扩展

1. 实验目的

（1）学习数码管的动态接口方法及编程方法；

2. 实验设备

计算机、Keil μVision 软件、Proteus 软件。

3. 实验内容

采用 C51 或汇编语言编程，在 Proteus 下仿真完成以下任务。

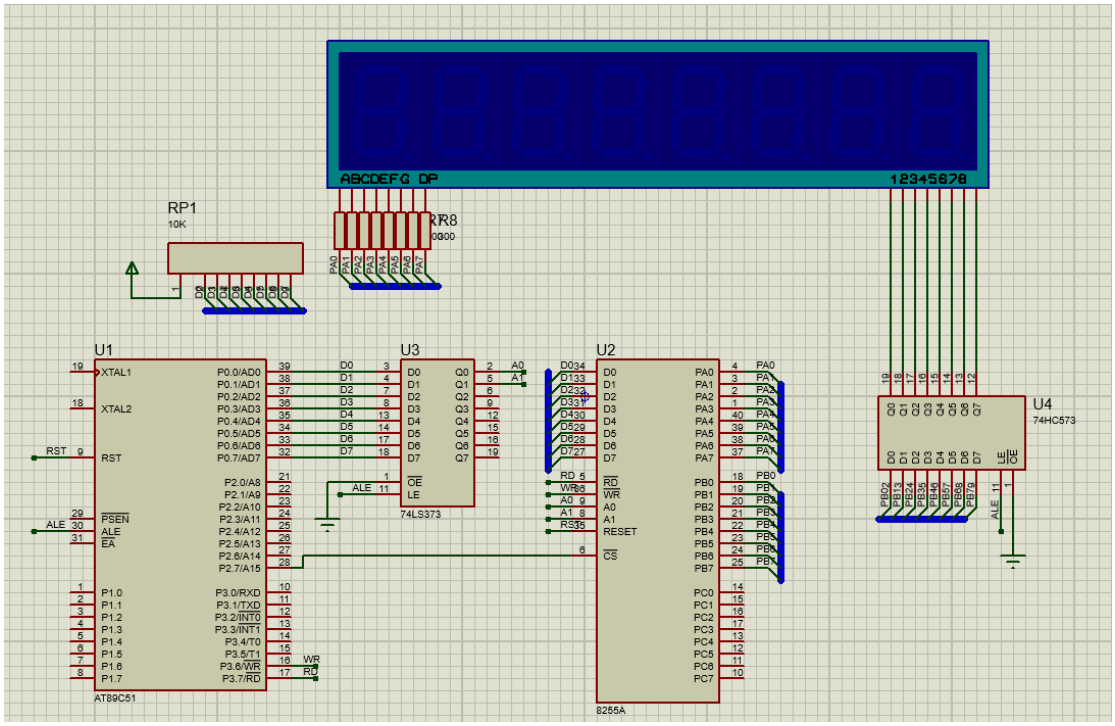
（1）数码管的动态显示

利用 8255A 的 A 口作为数码管的段控口，B 口作为数码管的位控口，编程使数码管显示 12345678。假设 f_{osc}＝11.0592MHz。

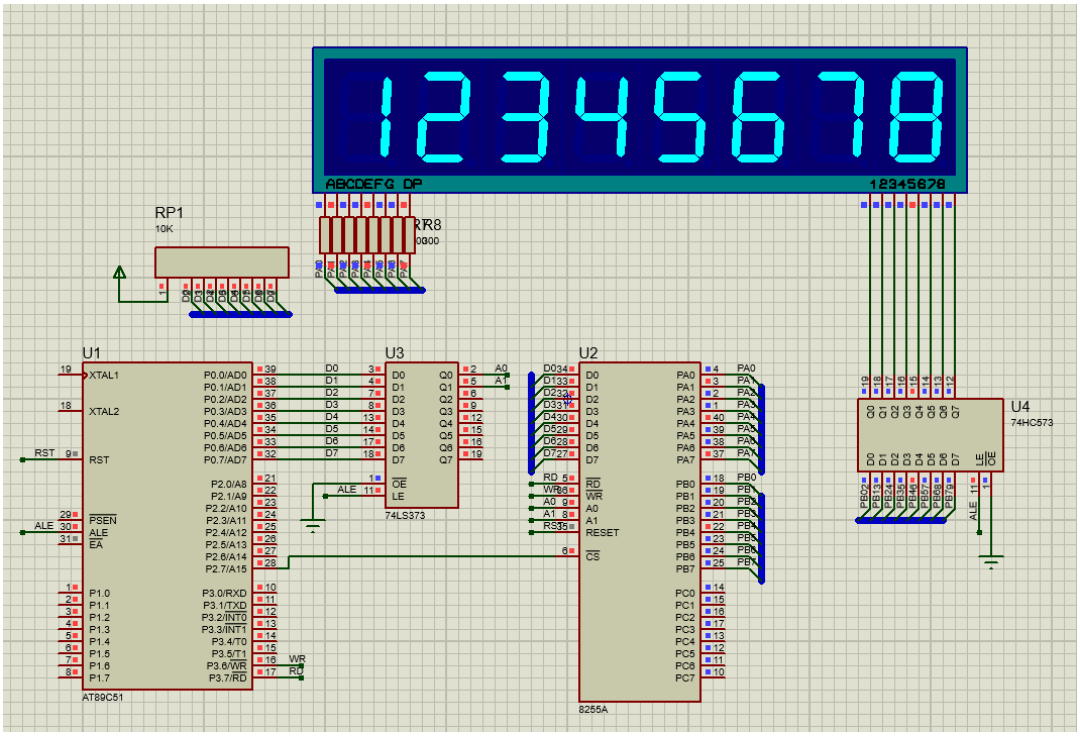
运行代码：

```
#include <reg51.h>
#include <absacc.h>
#include <intrins.h>

#define uchar unsigned char
#define uint unsigned int

#define portA XBYTE[0x7FFC]
#define portB XBYTE[0x7FFD]
#define portCTR XBYTE[0x7FFF]

uchar code table[]={0xf9,0xa4,0xb0,0x99,0x92,0x82,0xf8,0x80};

void delayms(uint i)
{
  uint j,k;
  for(j=i;j>0;j--)
    for(k=115;k>0;k--);
}

void main()
{
  uchar n,b=0x01;
  portCTR=0x80;
  portB=0x00;
  delayms(10);
  while(1)
  {
    for(n=0;n<8;n++)
    {
    portA=0xFF;
    portB=b;
    _nop_();_nop_();_nop_();
    portA=table[n];
    delayms(5);
    b=_crol_(b,1);
    }
  }
}
```

电路图：



运行结果：



4. 实验提示

（1）总线的画法：总线模式下绘制时，总线需要与器件引脚保持一

定距离，以便于器件引脚与总线的“分支线”绘制。为了和一般导线区分，一般用斜线来表示分支线。为了便于绘制，需要把“线路自动路径”功能关掉，也可在打开“线路自动路径”功能的时候按 Ctrl 键进行 45° 绘制。一条分支线画好后，双击引脚端点，可实现自动画线，所画线与第一条分支线平行。

（2）分支线的快捷命名

画好分支线后，务必要将分支线命名。一种方法是：右键单击分支线，在选中线段的同时会出现一个对话框，选择 Place wire label 可在 Label 下输入线的名称。另外一种快捷的命名方法如下：首先选中该分支线，然后在“工具”（Tools）下的“属性分配工具”（Property Assignment Tool）下命名，如图所示。假设总线的名称为 P0.0~P0.7，则在 String 处输入 NET=P0.#, “#”的含义是从 0 开始，增量为 1。点击“ok”后可自动命名。

（4）即使程序没有问题，Proteus 中仿真多位数码管时可能会出现乱码，达不到预期的显示效果，但是实际电路运行时可能没有问题。此时可以在 Proteus 下右击鼠标，修改数码管的激发时间试一试，看是否有效。数码管出现乱码或显示不正确是与 Proteus 软件本身有关。

（5）Proteus 中的数码管有两种类型，一种是 7 段 BCD 数码管（如 7SEG-BCD），其形状如图 11 所示。其输入值为 4 位的 BCD 码，显示为该 BCD 码对应的十进制数字。另外一种为普通的数码管，如 7 段共阳极数码管 7SEG-COM-ANODE（图 12），7 段共阴极绿色数码管

7SEG-COM-CAT-GRN 等。

5．观察与分析

（1）分析静态显示和动态显示方式各自的优缺点，指出各自的适用场合。

①静态显示

优点：显示稳定、清晰：静态显示是指将数字信号直接送入数码管，通过控制每个数码管的开关状态来显示数字。并且简单直接，不需要复杂的动态刷新控制，编程相对简单。

缺点：占用 IO 口线多：每个数码管的选段必须接入一个 8 位的数据线来保持显示字形码，编程复杂度较高：由于需要控制每个数码管的显示，编程时需要处理更多的细节。

适用场合：适用于显示固定的信息，例如温度计、电压表等，这些场合下不需要频繁更新显示内容，因此静态显示可以提供稳定的读数。

②动态显示

优点：节省 I/O 口资源：动态显示将所有数码管的段选并联在一起，由位选控制，控制那个数码管有效。通用性和可扩展性较好。

缺点：动态显示需要通过快速地切换多个数码管的开关状态来实现，需要进行频繁的动态刷新控制。容易产生闪烁和干扰，由于是分时段依次送入数码管中，容易产生闪烁和干扰，显示效果不够稳定、清晰。

适用场合：适用于需要显示多位数字或需要频繁更新信息的场合，例如计时器、计数器等，通过快速刷新来实现多位数字的同时显示。